Laboratorium 4 - Three.js: przejście z WebGL2 i import modeli 3D

1.Wprowadzenie do biblioteki Three.js

Three.js to biblioteka języka JavaScript przeznaczona do tworzenia i wyświetlania interaktywnych scen trójwymiarowych w przeglądarce internetowej. Umożliwia budowanie środowisk 3D, importowanie gotowych modeli, obsługę kamer, materiałów, oświetlenia, cieni oraz animacji. Jej zastosowanie obejmuje między innymi gry komputerowe, wizualizacje architektoniczne, symulacje i interaktywne prezentacje produktów.

Biblioteka Three.js wykorzystuje mechanizmy grafiki komputerowej poznane podczas wcześniejszych laboratoriów z WebGL2. Podstawowa różnica polega na sposobie zarządzania procesem renderowania. W WebGL2 konieczne jest samodzielne przygotowanie buforów wierzchołków (VBO), atrybutów (VAO), utworzenie shaderów i przekazywanie do nich macierzy transformacji i kolorów obiektów. W Three.js większość tych czynności jest realizowana automatycznie przez bibliotekę.

Nie oznacza to jednak, że Three.js zastępuje WebGL2 zupełnie inną technologią. Wykorzystując renderer WebGL, nadal korzystamy z procesora graficznego (GPU), shaderów oraz mechanizmów przekształcania geometrii. Biblioteka jedynie udostępnia wygodniejsze narzędzia do zarządzania tymi elementami.



2. Przygotowanie własnego projektu

2.1. Otwórz projekt z poprzedniego bloku WebGL w Visual Studio Code.

2.2. W terminalu katalogu zawierającego package.json zainstaluj bibliotekę i uruchom ponownie serwer:

```
npm install three
npm run dev
```

2.3.Pobierz i umieść modele low poly samochodu i drzewa w katalogu *public/models/*. W przykładach przyjmujemy nazwy **car.glb** i **tree.glb**. Jeżeli Twoje pliki mają inne nazwy, odpowiednio zmień ścieżki w kodzie. W przypadku formatu .gltf przenieś również wszystkie wykorzystywane pliki .bin i tekstury, zachowując ich ścieżki względne.

2.4. Utwórz w katalog *src/three/*, a w nim pliki *objects.js* i *modelLoader.js*. Na tym etapie pozostają puste.

3. Pierwsza scena Three.js- zmiana renderowania sceny w pliku main.js

3.1. W istniejącym *src/main.js*, dodaj import biblioteki

```
import * as THREE from 'three';
```

Następnie uporządkuj dotychczasowe importy, usuwając zależności związane z renderowaniem w WebGL2:

- Usuń import funkcji z */webgl/index.js*, odpowiedzialnych za tworzenie geometrii, obsługę shaderów i rysowanie obiektów.
- Usuń import mat4 z biblioteki gl-matrix, ponieważ transformacjami i kamerą będzie zarządzać Three.js.
- Zachowaj importy danych sceny (rows, scene) oraz funkcji odpowiedzialnych za logikę gry (updateCars, movePlayer, gameState, updateGameState, restartGame).

Zachowaj dotychczasowy kod odpowiedzialny za logikę gry, obsługę klawiatury, wyświetlanie informacji w HUD oraz istniejącą pętlę frame(). Usuń jedynie elementy związane z renderowaniem w WebGL2, które w kolejnych krokach zastąpimy rozwiązaniami Three.js.

3.2. Pozostając w pliku *src/main.js* odszukaj fragmenty kodu odpowiedzialne za przygotowanie i renderowanie sceny z wykorzystaniem WebGL2. Należy usunąć:

- Inicjalizację kontekstu WebGL2 (gl).
- Tworzenie programu shaderów (program) oraz pobieranie uniformów (uniforms).
- Tworzenie geometrii (planeMesh, cubeMesh).
- Funkcję resizeCanvas() odpowiedzialną za dostosowanie rozmiaru obszaru renderowania.
- Dotychczasową funkcję render() wraz z jej zawartością.

3.3. Bezpośrednio pod deklaracjami zmiennych odpowiadających za dostęp do elementów HTML (canvas, scoreLabel itd.), dodaj poniższy kod:

```
const scene3D = new THREE.Scene();
scene3D.background = new THREE.Color(0x141b26);
const renderer3D = new THREE.WebGLRenderer({ canvas, antialias: true });
renderer3D.setPixelRatio(Math.min(window.devicePixelRatio || 1, 2));

const camera3D = new THREE.OrthographicCamera(-3, 3, 2.2, -2.2, 0.1, 100);
camera3D.position.set(0, 2.4, 3.2);
camera3D.lookAt(0, -0.2, -0.35);
```

Jeżeli w pliku pozostał obiekt camera z poprzednich laboratoriów, usuń go. Zastępuje go nowa kamera camera3D.

3.4. Utworzenie pierwszego obiektu 3D.

W Three.js obiekty trójwymiarowe tworzymy za pomocą klasy Mesh, która łączy dwa podstawowe elementy:

- **Geometria (BufferGeometry)**- określa kształt i wymiary obiektu. Three.js udostępnia gotowe geometrie, np. BoxGeometry (prostopadłościan) i SphereGeometry (kula).
- **Materiał (Material)** - określa wygląd powierzchni obiektu, np. kolor, teksturę oraz sposób reagowania na oświetlenie.

W pliku *src/main.js*, bezpośrednio po utworzeniu kamery camera3D, dodaj poniższy kod tworzący żółty sześcian:

```
const testBox = new THREE.Mesh(
  new THREE.BoxGeometry(0.25, 0.25, 0.25),
  new THREE.MeshBasicMaterial({ color: 0xffbf20 })
);
testBox.position.set(0, -0.25, 0.70);
scene3D.add(testBox);
```

W powyższym przykładzie:

- BoxGeometry(0.25, 0.25, 0.25)- tworzy sześcian o wymiarach wynoszących 0.25 jednostki.
- MeshBasicMaterial({color: 0xffbf20})- nadaje obiektowi żółty kolor zapisany w formacie szesnastkowym.
- position.set(0, -0.25, 0.70)- ustawia położenie sześcianu w przestrzeni 3D, kolejno w osiach X, Y i Z.
- scene3D.add(testBox)- dodaje utworzony obiekt do sceny, dzięki czemu będzie mógł zostać wyświetlony przez renderer.

3.5. W poprzednich krokach utworzyliśmy scenę, kamerę, renderer oraz pierwszy obiekt 3D. Aby wyświetlić przygotowaną scenę w przeglądarce, musimy teraz połączyć te elementy w funkcji *render()*. Bezpośrednio pod kodem tworzącym obiekt testBox, dodaj nową funkcję *render()*.

```
function render() {
  const w = Math.max(1, canvas.clientWidth);
  const h = Math.max(1, canvas.clientHeight);
  renderer3D.setSize(w, h, false);
  const zoom = 2.2, aspect = w / h;
  camera3D.left = -zoom * aspect; camera3D.right = zoom * aspect;
  camera3D.top = zoom; camera3D.bottom = -zoom;
  camera3D.updateProjectionMatrix();
  renderer3D.render(scene3D, camera3D);
}
```

Funkcja render() dostosowuje rozmiar renderowania do aktualnych wymiarów elementu canvas za pomocą clientWidth, clientHeight i renderer3D.setSize(). Następnie oblicza proporcje obrazu (aspect) oraz wielkość widocznego obszaru (zoom), na podstawie których ustala granice kamery ortograficznej. Metoda updateProjectionMatrix() aktualizuje ustawienia projekcji, a

renderer3D.render() wyświetla scenę z perspektywy wybranej kamery. W przeciwieństwie do WebGL2 biblioteka Three.js automatycznie obsługuje większość operacji związanych z renderowaniem, takich jak przekazywanie macierzy do shaderów i rysowanie obiektów.

4. Import własnych modeli GLB/glTF

Dotychczas obiekty w naszej grze tworzyliśmy z prostych brył. Three.js umożliwia import gotowych modeli 3D wraz z ich geometrią, materiałami, teksturami i hierarchią.

glTF (GL Transmission Format) jest otwartym formatem wymiany scen 3D, przygotowanym m.in. do wydajnego przesyłania modeli do aplikacji internetowych. Obsługuje siatki geometryczne, hierarchię, materiały PBR, tekstury oraz animacje.

GLB to binarny wariant glTF 2.0, w którym geometria, materiały i obrazy zazwyczaj mieszczą się w jednym pliku. Plik `.gltf` jest tekstowym plikiem JSON i może wskazywać zasoby .bin i tekstury przechowywane obok niego. W Three.js oba warianty wczytuje się przez GLTFLoader.

Pliki wczytujemy bez ręcznego parsowania wierzchołków, tworzenia VBO i VAO ani odtwarzania materiałów w shaderach. Dostajemy całe drzewo obiektów, które można pozycjonować, obracać i skalować w Three.js

4.1. W **src/three/modelLoader.js** utwórz pierwszy moduł Three.js. Zaimportuj GLTFLoader i napisz funkcję wczytującą plik.

```
import * as THREE from 'three';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

const loader = new GLTFLoader();
export async function loadTemplate(path) {
   const gltf = await loader.loadAsync(path);
   return gltf.scene;
}
```

4.2. Modele mogą różnić się rozmiarem oraz położeniem punktu odniesienia. Aby ułatwić ich rozmieszczanie na planszy, przygotujemy funkcję makeInstance(template, targetSize), która utworzy kopię modelu i dostosuje jego wymiary.

4.2.1. W pliku *modelLoader.js* rozpocznij tworzenie funkcji *makeInstance()*. Dodaj kod kopiujący model, wyznaczający jego wymiary i dostosowujący skalę do wartości targetSize. Na tym etapie nie zamykaj jeszcze funkcji.

```
export function makeInstance(template, targetSize) {
   const model = template.clone(true);

   let box = new THREE.Box3().setFromObject(model);
   const size = box.getSize(new THREE.Vector3());
   const largest = Math.max(size.x, size.y, size.z);

   if (largest <= 0) throw new Error('Pusty model 3D');

   model.scale.multiplyScalar(targetSize / largest);
```

```
  model.updateMatrixWorld(true);
```

W dalszej części tej samej funkcji dodaj kod ponownie obliczający granice modelu. Następnie wyśrodkuj go w osiach X i Z, ustaw jego podstawę na wysokości Y = 0 i umieść w grupie THREE.Group. Zakończ funkcję instrukcją return root; oraz nawiasem zamykającym.

```
box = new THREE.Box3().setFromObject(model);
model.position.x -= (box.min.x + box.max.x) / 2;
model.position.y -= box.min.y;
model.position.z -= (box.min.z + box.max.z) / 2;
const root = new THREE.Group();
root.add(model);
return root; // zwrócenie punktu odniesienia hierarchii całej grupy
}
```

Kod wyśrodkowuje model w osiach X i Z oraz ustawia jego podstawę na wysokości Y = 0. Następnie umieszcza go w grupie THREE.Group, dzięki której możemy przesuwać, obracać i skalować cały model jako jeden obiekt.

4.3.Wczytywanie modelu samochodu w pliku głównym **main.js**

4.3.1. Przygotowanie oświetlenia sceny

W pliku *src/main.js* dodaj import funkcji *loadTemplate()* i *makeInstance()* z utworzonego wcześniej modułu *modelLoader.js*:

```
import { loadTemplate, makeInstance } from './three/modelLoader.js';
```

Zaimportowane modele mogą wykorzystywać materiały reagujące na światło. Dlatego przed ich wyświetleniem dodamy dwa podstawowe źródła oświetlenia. Bezpośrednio po utworzeniu sceny i kamery camera3D dodaj:

```
scene3D.add(new THREE.AmbientLight(0xffffff, 1.5));
const light = new THREE.DirectionalLight(0xffffff, 2.0);
light.position.set(-2, 5, 4);
scene3D.add(light);
```

AmbientLight równomiernie rozjaśnia obiekty, natomiast **DirectionalLight** oświetla je z określonego kierunku, pomagając uwidocznić ich trójwymiarowy kształt.

4.3.2. Import i wyświetlenie modelu samochodu

W pliku main.js, przed uruchomieniem istniejącej pętli animacji frame(), dodaj kod wczytujący model samochodu:

```
const carTemplate = await loadTemplate('/models/car.glb');
const previewCar = makeInstance(carTemplate, 0.40);
previewCar.position.set(0, -0.45, 0);
scene3D.add(previewCar);
scene3D.remove(testBox);
```

Funkcja loadTemplate() wczytuje model, a makeInstance() tworzy jego kopię i dopasowuje rozmiar. Następnie ustawiamy pozycję samochodu, dodajemy go do sceny i usuwamy wcześniejszy sześcian.

Uruchom projekt. Zamiast żółtego sześcianu powinien pojawić się model samochodu. Sprawdź, czy ma odpowiednią skalę, pozycję i orientację.

Eksperyment 1

W pliku main.js, po utworzeniu modelu previewCar, dodaj instrukcję obracającą samochód wokół osi Y:

```
previewCar.rotation.y = Math.PI / 2;
```

Uruchom projekt i sprawdź, jak zmieniła się orientacja samochodu. Następnie poeksperymentuj z innymi kątami obrotu. Zmień również wartość 0.40 w wywołaniu makeInstance() i sprawdź, jak wpływa ona na wielkość modelu. Po zakończeniu eksperymentu przywróć pierwotny rozmiar i rotację obiektu.

4.4. W pliku main.js wczytaj model drzewa, korzystając z wcześniej przygotowanych funkcji. Następnie ustaw samochód i drzewo obok siebie, aby porównać ich rozmiary i położenie.

```
const treeTemplate = await loadTemplate('/models/tree.glb');
const previewTree = makeInstance(treeTemplate, 0.70);
previewCar.position.x = -0.45;
previewTree.position.set(0.45, -0.45, 0);
scene3D.add(previewTree);
```

Uruchom projekt i sprawdź, czy oba modele są poprawnie wyświetlane.

Eksperyment 2- analiza złożoności modeli 3D

Modele 3D składają się z trójkątów, których liczba wpływa na ilość geometrii przetwarzanej podczas renderowania. Większa liczba trójkątów nie zawsze oznacza jednak lepszy wygląd modelu, dlatego warto kontrolować ich złożoność.

1. W *modelLoader.js* dopisz funkcję zliczającą trójkąty w siatkach modelu:

```
export function countTriangles(model) {
  let triangles = 0;
  model.traverse(part => {
    if (!part.isMesh) return;
    const g = part.geometry;
    triangles += (g.index ? g.index.count : g.attributes.position.count) / 3;
  });
  return triangles;
}
```

2. W pliku main.js zaimportuj funkcję countTriangles() i wyświetl w konsoli liczbę trójkątów samochodu oraz drzewa:

```
console.log('Samochód:', countTriangles(carTemplate));
console.log('Drzewo:', countTriangles(treeTemplate));
```

3. Uruchom projekt i sprawdź wyniki w konsoli przeglądarki (F12). Porównaj liczbę trójkątów oraz rozmiary plików GLB. Zastanów się, jak złożoność modeli może wpływać na wydajność gry, szczególnie gdy na scenie znajduje się wiele obiektów.

Pamiętaj, że porównywane modele przedstawiają różne obiekty, dlatego sama liczba trójkątów nie pozwala ocenić, który jest lepiej wykonany.

5. Utworzenie planszy i postaci gracza

W tej części odtworzymy planszę i postać gracza za pomocą Three.js. Do rozmieszczenia pasów trawy i drogi wykorzystamy istniejące dane z pliku *sceneData.js,* natomiast postać gracza utworzymy z prostych brył geometrycznych. W kolejnych krokach połączymy nowe obiekty z dotychczasową logiką gry.

5.1. Przygotowanie modułów obiektów

Utwórz plik *src/three/objects.js*. Zaimportuj Three.js, dane gry oraz funkcję makeInstance():

```
import * as THREE from 'three';
import { rows, scene as gameScene } from '../game/sceneData.js';
import { makeInstance } from './modelLoader.js';

export const GROUND_Y = -0.45;

export const objects = {
  playerRoot: null,
  cars: [],
  trees: []
};
```

Stała GROUND_Y określa wysokość podłoża, natomiast objects będzie przechowywać obiekty Three.js, którymi będziemy sterować podczas gry.

5.2. Utworzenie pasów trawy i drogi

W tym samym pliku rozpocznij funkcję createBasicObjects(scene3D), która będzie tworzyć podstawowe elementy planszy:

```
export function createBasicObjects(scene3D) {
  const rowGeometry = new THREE.BoxGeometry(1.8, 0.04, 0.65);

  for (const row of rows) {
    const color = row.type === 'road' ? 0x30323a : 0x2ba344;

    const rowMesh = new THREE.Mesh(
      rowGeometry,
      new THREE.MeshBasicMaterial({ color })
    );
    rowMesh.position.set(0, GROUND_Y - 0.02, row.z);
    scene3D.add(rowMesh);
  }
```

Każdy pas powstaje na podstawie danych z sceneData.js. Kolor szary oznacza drogę, a zielony trawę, zachowując układ przygotowany podczas wcześniejszych laboratoriów.

5.3. Utworzenie postaci gracza

W dalszej części funkcji createBasicObjects() utwórz żółtą postać złożoną z dwóch brył:

```
  const playerRoot = new THREE.Group();
  const yellow = new THREE.MeshBasicMaterial({ color: 0xffbf20 });

  const body = new THREE.Mesh(
    new THREE.BoxGeometry(0.18, 0.24, 0.18), yellow
  );

  const head = new THREE.Mesh(
    new THREE.BoxGeometry(0.13, 0.13, 0.13), yellow
  );

  body.position.y = 0.12;
  head.position.y = 0.30;
  playerRoot.add(body, head);
  scene3D.add(playerRoot);
  objects.playerRoot = playerRoot;
}
```

Obie bryły umieszczamy w grupie **playerRoot**, dzięki czemu będziemy mogli przesuwać całą postać jako jeden obiekt, bez konieczności osobnego aktualizowania położenia głowy i tułowia.

5.4. Wyświetlenie planszy i postaci

W pliku main.js dodaj import:

```
import { createBasicObjects } from './three/objects.js';
```

Następnie, po utworzeniu scene3D i przygotowaniu jej podstawowych ustawień wywołaj:

```
createBasicObjects(scene3D);
```

Uruchom projekt i sprawdź, czy wyświetlają się pasy trawy, drogi oraz żółta postać. W kolejnych krokach połączymy utworzone obiekty z dotychczasową logiką gry.

6. Wielokrotne wykorzystanie modeli

Dotychczas samochód i drzewo wyświetlaliśmy jedynie jako obiekty testowe. Teraz wykorzystamy zaimportowane modele do odtworzenia wszystkich pojazdów i drzew znajdujących się na planszy. Ich pozycje będziemy pobierać z istniejących danych gry, dzięki czemu zachowamy dotychczasową logikę ruchu.

6.1. Dodanie samochodów i drzew

W pliku objects.js dodaj funkcję addModels(), która utworzy samochody i drzewa na podstawie wcześniej wczytanych modeli GLB:

```
export function addModels(scene3D, carTemplate, treeTemplate) {
  for (const car of gameScene.cars) {
    const root = makeInstance(carTemplate, 0.40);

    root.rotation.y = car.speed > 0
      ? Math.PI / 2
      : -Math.PI / 2;

    scene3D.add(root);
    objects.cars.push(root);
  }

  for (const tree of gameScene.trees) {
    const root = makeInstance(treeTemplate, 0.70);

    scene3D.add(root);
    objects.trees.push(root);
  }
}
```

Funkcja tworzy osobny obiekt dla każdego samochodu i drzewa, bez ponownego wczytywania plików GLB. Zaimportowany model jest początkowo skierowany wzdłuż osi Z, natomiast samochody poruszają się wzdłuż osi X. Dlatego obracamy je o 90° wokół osi Y. Znak prędkości car.speed określa kierunek jazdy, a tym samym orientację pojazdu. W zależności od początkowej orientacji modelu GLB może być konieczne odwrócenie znaków kątów.

6.2. Zastąpienie modeli testowych

W pliku main.js usuń wcześniej wyświetlane modele podglądowe i utwórz właściwe obiekty gry:

```
scene3D.remove(previewCar, previewTree);
addModels(scene3D, carTemplate, treeTemplate);
```

6.3. Aktualizacja położenia obiektów

Samo dodanie modeli do sceny nie powoduje jeszcze ich poruszania. Aby samochody i postać reagowały na dotychczasową logikę gry, musimy aktualizować ich pozycje.

W pliku *objects.js* dodaj funkcję *syncObjects()*:

```
export function syncObjects() {
  // Pozycja gracza
  const p = gameScene.player.position;

  objects.playerRoot.position.set(
```

```
        p[0], GROUND_Y + p[1], p[2]
    );

    // Pozycje samochodów
    gameScene.cars.forEach((car, i) => {
        if (!objects.cars[i]) return;

        const v = car.position;
        objects.cars[i].position.set(
            v[0], GROUND_Y + v[1], v[2]
        );
    });

    // Pozycje drzew
    gameScene.trees.forEach((tree, i) => {
        if (!objects.trees[i]) return;

        const v = tree.position;
        objects.trees[i].position.set(
            v[0], GROUND_Y + v[1], v[2]
        );
    });
}
```

Funkcja odczytuje aktualne współrzędne z *sceneData.js* i przypisuje je odpowiednim obiektom Three.js. Dzięki temu zachowujemy dotychczasowe sterowanie graczem oraz ruch samochodów.

6.4. Połączenie z pętlą renderowania

W pliku *main.js* uzupełnij import:

```
import {createBasicObjects, addModels, syncObjects} from './three/objects.js';
```

Następnie w istniejącej funkcji *render()* wywołaj *syncObjects()* bezpośrednio przed narysowaniem sceny:

```
syncObjects();
renderer3D.render(scene3D, camera3D);
```

Uruchom projekt. Sprawdź, czy samochody poruszają się po planszy, a postać reaguje na klawisze strzałek. Zwróć uwagę, że zmieniliśmy sposób wyświetlania obiektów, ale nadal korzystamy z logiki gry przygotowanej podczas laboratoriów WebGL2.

Eksperyment 3.

W funkcji addModels() zmień rozmiar samochodów z 0.40 na 0.70 w wywołaniu makeInstance(). Uruchom grę i sprawdź, czy kolizje są wykrywane zgodnie z nowymi wymiarami pojazdów.

Zwróć uwagę, że wielkość widocznego modelu nie musi odpowiadać obszarowi wykrywania kolizji. W pliku gameRules.js kolizje sprawdzane są na podstawie wcześniej ustalonych wymiarów, które nie zmieniają się automatycznie wraz ze skalą modelu.

Po zakończeniu eksperymentu przywróć rozmiar samochodów do 0.40.

7. Zadania do sprawozdania

Zadanie 1. Własna postać gracza.

Zastąp żółtą postać modelem 3D wybranego zwierzęcia, pozyskanym samodzielnie w formacie GLB lub glTF. Dostosuj jego rozmiar, położenie i orientację tak, aby poprawnie wyświetlał się na planszy. Zachowaj dotychczasowe sterowanie oraz wykrywanie kolizji.

Zadanie 2. Kierunek poruszania postaci.

Dodaj możliwość automatycznej zmiany orientacji modelu gracza podczas poruszania się po planszy. Postać powinna obracać się w odpowiednim kierunku po naciśnięciu każdej ze strzałek, bez zmiany dotychczasowego sposobu przemieszczania.

Zadanie 3. Droga dwupasmowa.

Zmodyfikuj planszę, dzieląc dotychczasową drogę na dwa oddzielne pasy ruchu. Na pierwszym pasie samochody powinny poruszać się wyłącznie w prawo, natomiast na drugim wyłącznie w lewo. Zachowaj dotychczasowe sterowanie graczem oraz wykrywanie kolizji.

Wymagania dotyczące sprawozdania

Sprawozdanie powinno być podzielone na poszczególne laboratoria, z wyraźnym oznaczeniem **numeru laboratorium oraz numerów wykonanych zadań**.

Dla każdego zadania należy zamieścić:

1. **Opis rozwiązania** - krótkie wyjaśnienie, w jaki sposób wykonano zadanie, np. poprzez przedstawienie zmodyfikowanego fragmentu kodu wraz z krótkim komentarzem.
2. **Zrzut ekranu**- przedstawiający efekt wprowadzonych zmian na renderowanej scenie.

Po zakończeniu trzech laboratoriów z Three.js należy przesłać **jedno zbiorcze sprawozdanie**, zawierające rozwiązania zadań ze wszystkich trzech spotkań.